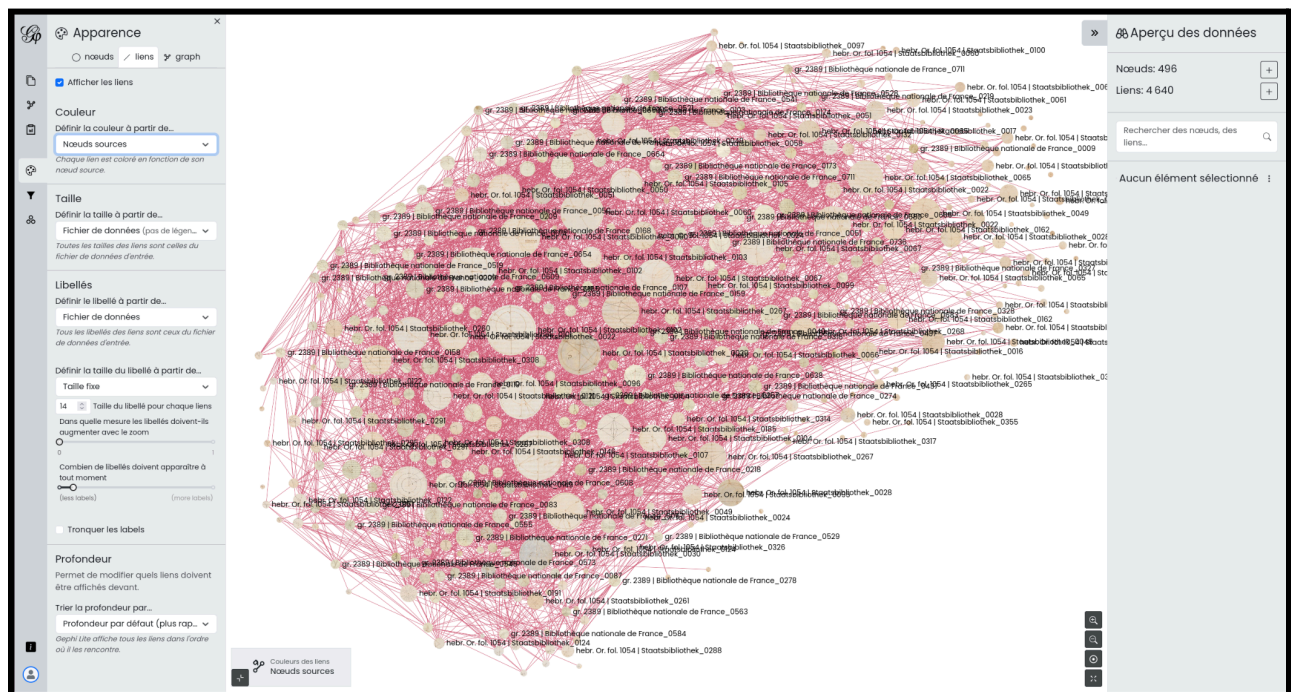
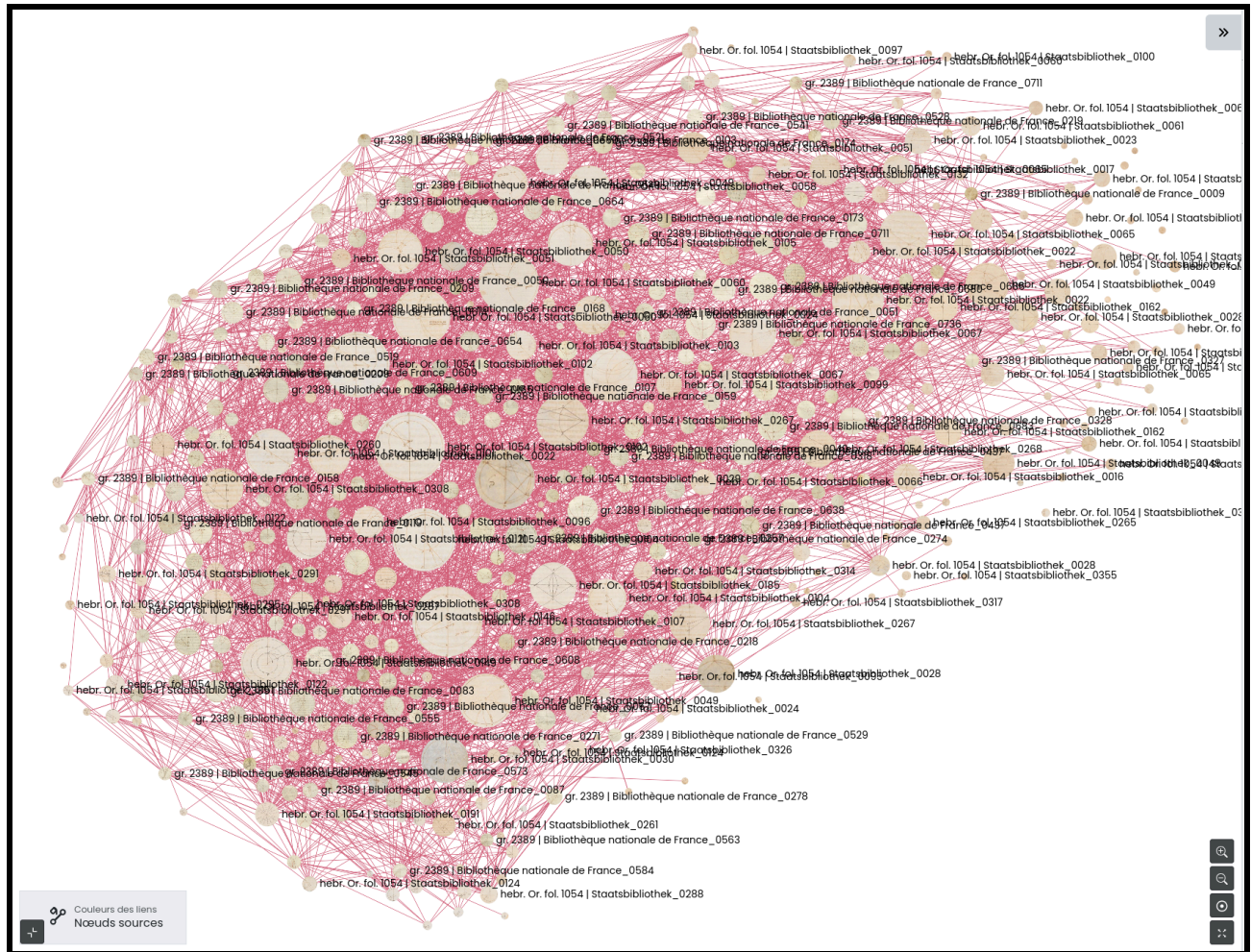


Documentation

Visualisations – 2025

Visualisation de type “Force-directed layout” - EIDA





1. Faire un export de la table region_pairs de la base de données EIDA
2. Nettoyer les données pour créer deux fichiers nodes_gephi.csv et edges_gephi.csv avec un script python
 - a. Importer les bibliothèques Python nécessaires : pandas pour analyser et manipuler les données et re pour utiliser des expressions régulières :

```
import pandas as pd
import re
```

- b. Importer le fichier csv avec la fonction pd.read_csv(). Ne pas oublier d'ajouter index_col=False pour que le script ne cherche pas une colonne devant servir d'index :

```
pd.read_csv('region-pairs.csv', index_col=False)
```

- c. Stocker le contenu du fichier dans un DataFrame nommé *witnesses* :

```
witnesses = pd.read_csv('region-pairs.csv', index_col=False)
```

- d. Choisir deux witnesses et extraire leurs *region_pairs* du fichier *region_pairs.csv* :

- Créer une variable avec les numéros des witnesses dont nous voulons extraire les *region_pairs*. Ici, nous avons choisi le witness 40 et le witness 1.

```
values = ["wit40", "wit1"]
```

- Créer une variable *pattern* pour construire une expression régulière qui détectera si une chaîne de caractère commence par un ou plusieurs mots parmi les values suivant d'un *_*.

```
pattern = r"^(" + "|".join([re.escape(v) + r"_" for v in values]) + ")"
```

- Ajouter deux variables *img_1* et *img_2* pour vérifier dans chaque ligne du DataFrame si la valeur des colonnes *img_1* et *img_2* commence par l'un des préfixes définis dans la variable *pattern* :

```
img1 = witnesses["img_1"].str.contains(pattern, na=False)  
img2 = witnesses["img_2"].str.contains(pattern, na=False)
```

- Créer deux variables contenant des séries de booléens indiquant pour chaque ligne si la colonne *img_1* ET (uniquement si les deux conditions sont vraies) la colonne *img_2*.

```
chosen_witnesses = witnesses[img1 & img2]
```

- Vérifier le nombre de lignes de notre DataFrame conservées dans la variable *chosen_witnesses* grâce à la fonction `len()`.

```
lines_number = len(chosen_witnesses)
lines_number
```

e. Nettoyer les données dans les witnesses sélectionnés :

- Pour supprimer les colonnes de notre tableau dont nous n'avons pas besoin, il faut d'abord créer une liste contenant les noms des colonnes à supprimer :

```
columns_suppression = ["regions_id_1", "regions_id_2", "created_at", "updated_at"]
```

- Supprimer les colonnes de la liste *columns_suppression* grâce à la méthode `.drop()` :

```
new_witnesses = chosen_witnesses.drop(columns=columns_suppression)
```

- Exporter les résultats dans un fichier csv en utilisant `pd.to_csv()` :

```
new_witnesses.to_csv('region_pairs_nettoyees.csv', index=False)
```

f. Créer un fichier pour les nodes :

- Extraire toutes les regions depuis `img_1` et `img_2`. Supprimer les doublons et valeurs inutiles. Générer un id unique pour chaque region. Exporter le résultat dans un fichier temporaire csv :

```
regions = pd.concat([new_witnesses['img_1'], new_witnesses['img_2']])

regions_unique = regions.dropna().drop_duplicates().reset_index(drop=True)

nodes = pd.DataFrame({
    'id': range(1, len(regions_unique) + 1), # ids 1, 2, 3, ...
    'region': regions_unique
})
```



```
nodes.to_csv('nodes.csv', index=False)
```

- Ajouter trois colonnes supplémentaires à notre DataFrame.
Ajouter la cote du witness dans une nouvelle colonne :

```
def assign_cote(region):  
    if region.startswith('wit1'):  
        return "gr. 2389 | Bibliothèque nationale de France"  
    elif region.startswith('wit40'):  
        return "hebr. Or. fol. 1054 | Staatsbibliothek"  
    else:  
        return ""
```

```
nodes['cote'] = nodes['region'].apply(assign_cote)
```

```
print(nodes)
```

```
nodes.to_csv('nodes_2.csv', index=False)
```

- Extraire le numéro de page pour chaque regions (numero_de_page). Il s'agit du troisième élément pour chaque nom de fichier image :

```
def extraire_page(region):  
    try:  
        parts = region.split('_')  
        return parts[2] # 3e élément, numéro de page  
    except IndexError:  
        return ''
```

```
nodes['numero_de_page'] = nodes['region'].apply(extraire_page)
```

```
nodes.to_csv('nodes_3.csv', index=False)
```

- Créer une nouvelle colonne label en combinant les colonnes cote et numero_de_page :

```
nodes['label'] = nodes['cote'].fillna('') + '_' +  
nodes['numero_de_page'].fillna('')  
nodes['label'] = nodes['label'].str.strip('_')  
  
nodes.to_csv('nodes_4.csv', index=False)
```

g. Créer un fichier pour les arêtes :

- Créer une variable avec le nom des colonnes du csv d'origine à supprimer et exporter le résultat dans le fichier aretes.csv :

```
colonnes_a_supprimer = ['category', 'category_x', 'similarity_type']

aretes = new_witnesses.drop(columns=colonnes_a_supprimer)

aretes.to_csv('aretes.csv', index=False)
```

h. Créer les deux fichiers nodes et edges finaux :

```
#Charger les nœuds (nodes) depuis nodes_4.csv
new_witnesses = pd.read_csv('nodes_4.csv')

#Charger les arêtes (edges) depuis le fichier aretes.csv
aretes = pd.read_csv('aretes.csv')

#Construire un dictionnaire region -> id pour faire la correspondance
region_to_id = dict(zip(new_witnesses['region'], new_witnesses['id']))

# Remplacer les noms dans img_1 et img_2 par leurs ids correspondants
aretes['Source'] = aretes['img_1'].map(region_to_id)
aretes['Target'] = aretes['img_2'].map(region_to_id)

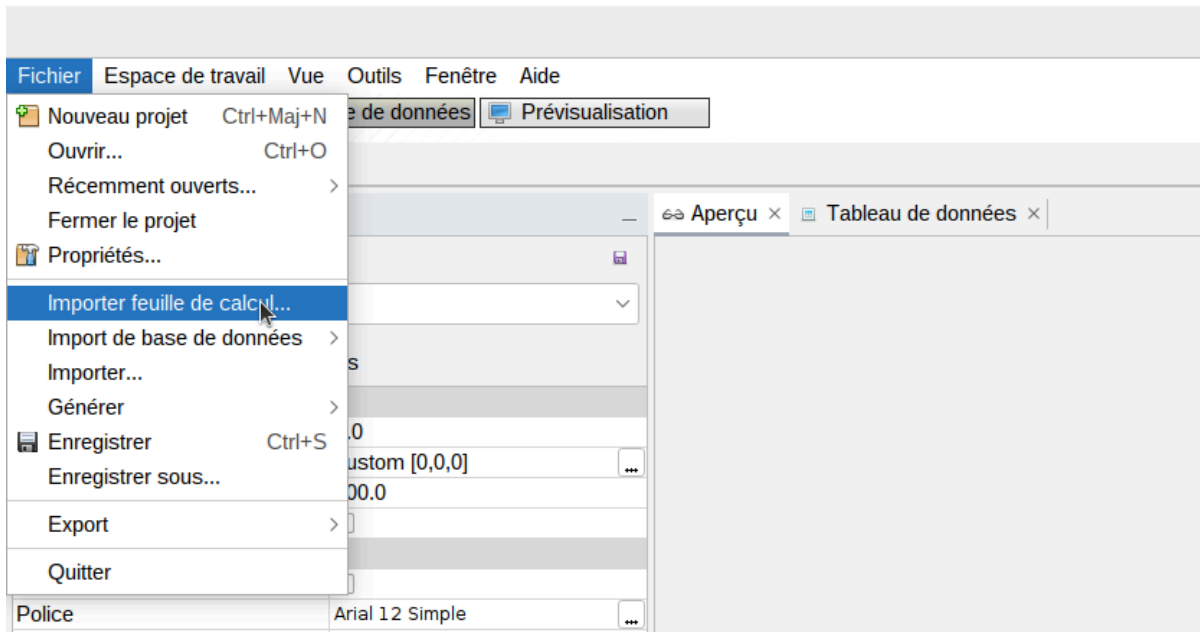
# Supprimer les lignes où Source ou Target sont NaN (car pas de correspondance)
aretes_clean = aretes.dropna(subset=['Source', 'Target']).copy()

# Ajouter une colonne Weight (poids) = 1 pour Gephi
aretes_clean['Weight'] = 1

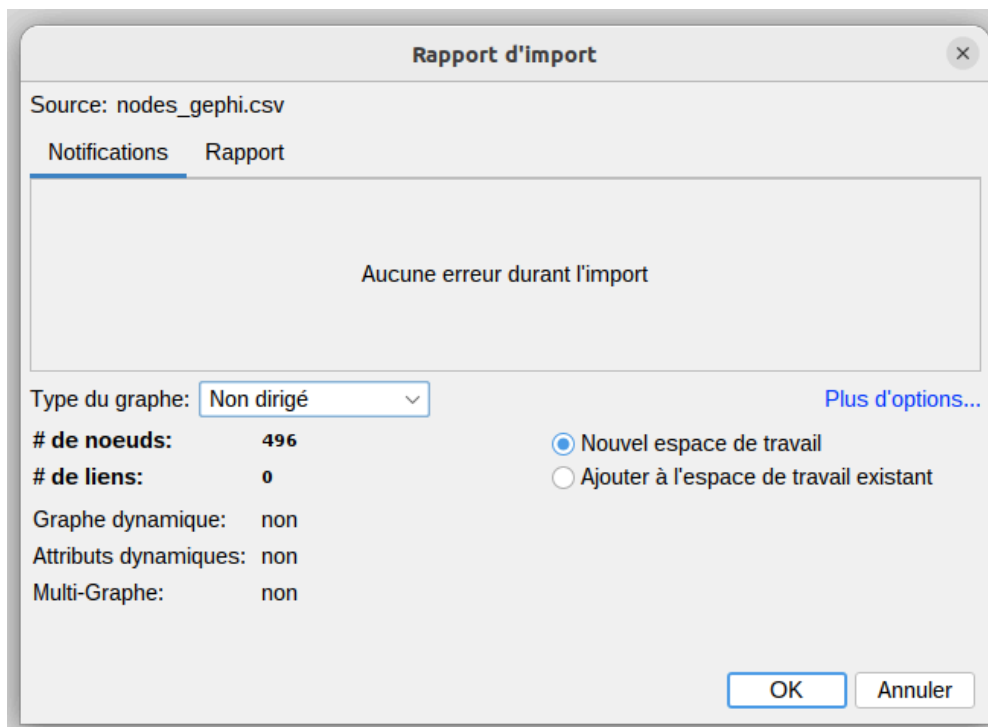
#Exporter les fichiers au format CSV pour Gephi
new_witnesses[['id', 'region', 'cote', 'numero_de_page', 'label']].to_csv('nodes_gephi.csv', index=False)
aretes_clean[['Source', 'Target', 'Weight']].to_csv('edges_gephi.csv', index=False)

print("Fichiers nodes_gephi.csv et edges_gephi.csv créés pour Gephi.")
```

3. Importer les deux fichiers nodes_gephi.csv et edges_gephi.csv dans Gephi en local (la version gephi-0.9.6) :



4. Choisir un type de graphe non-dirigé et mettre les deux fichiers csv dans le même espace de travail :



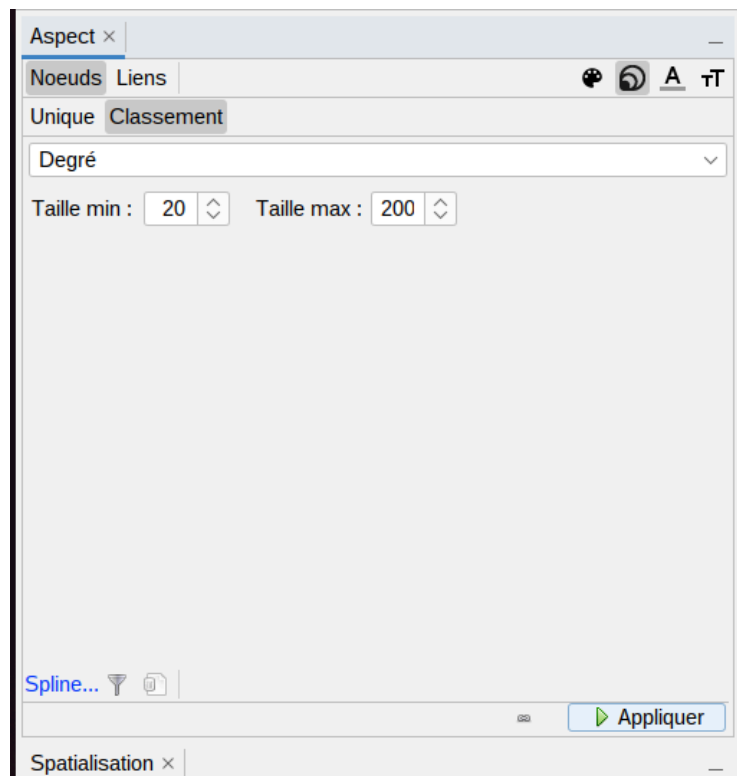
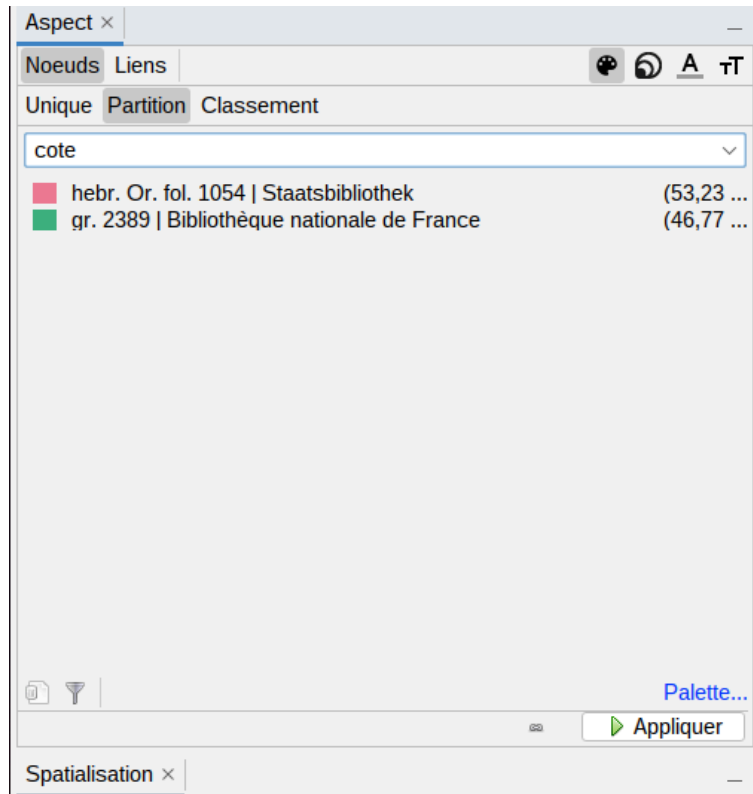
5. Regarder leur contenu dans le Data laboratory :

6. Choisir la spatialisation Force Atlas et modifier la Force de répulsion :

7. Dans Statistiques calculer le Degré et le Degré pondéré :

Filtres Statistiques x		
Paramètres		
Vue générale du réseau		
Degré	50,774	Exécuter
Degré pondéré	50,774	Exécuter
Diamètre		Exécuter
Densité		Exécuter
HITS		Exécuter
PageRank		Exécuter
Composantes Connexes		Exécuter
Community Detection		
Modularité		Exécuter
Statistical Inference		Exécuter
Vue générale des noeuds		
Coefficient de Clustering		Exécuter
Centralité Eigenvector		Exécuter
Vue générale des liens		
Plus courts chemins		Exécuter
Dynamique		
# Noeuds		Exécuter
# Liens		Exécuter
Degré		Exécuter
Coefficient de clustering		Exécuter

8. Dans Aspect, choisir cote dans dans Partition pour la couleur des nodes et Degré dans Classement pour la taille des nodes :



9. Choisir Déchevauchement dans Spatialisation :

Spatialisation ×

Déchevauchement

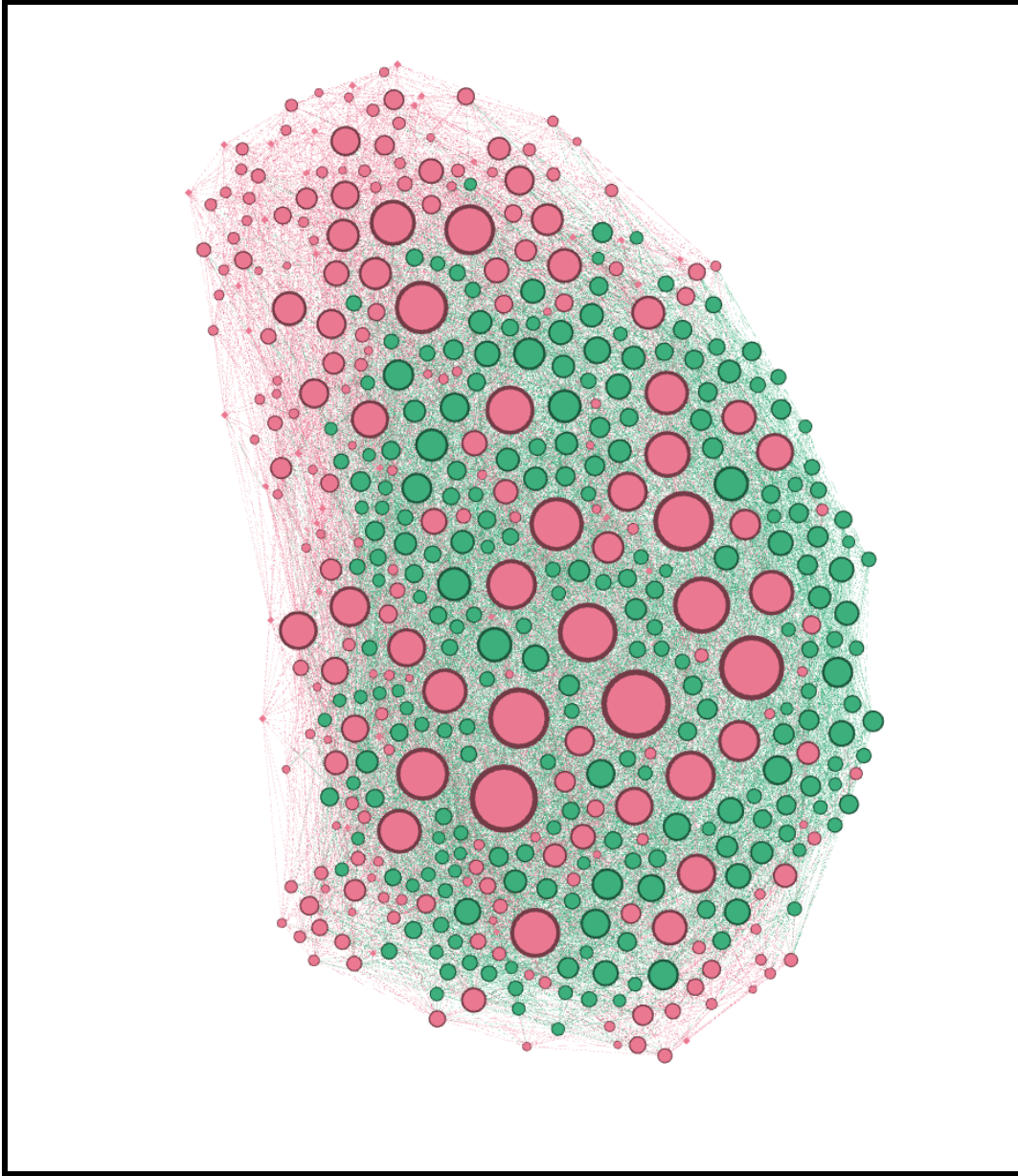
Exécuter

▼ Noverlap

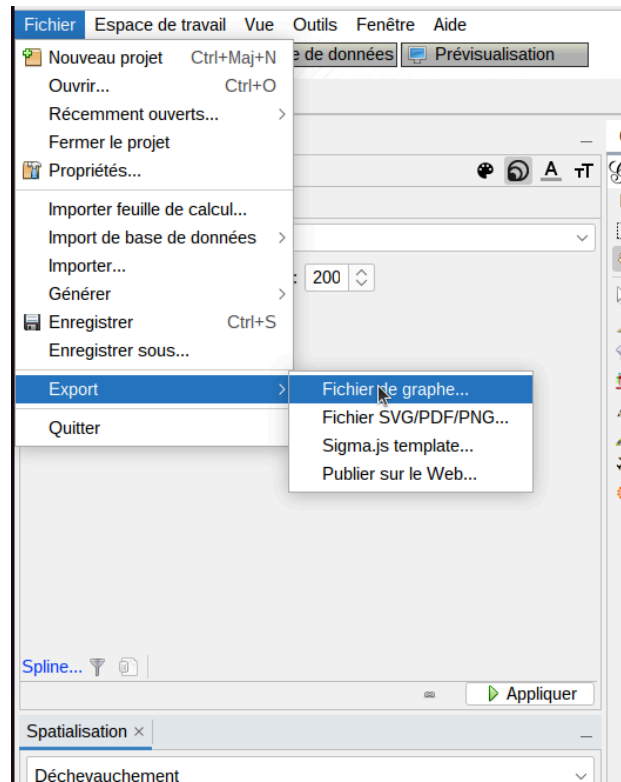
speed	3.0
ratio	1.2
margin	5.0

Déchevauchement

10. Voici le résultat :



11. Faire un export en fichier graphe .gexf.



Nom du fichier:

Type de fichier: Fichiers GEXF (*.gexf)

Enregistrer Annuler Options...

Graphe: ☒ **Complet** Le graphe complet est exporté

☐ **Visible** Seul le graphe visible est exporté

12. Nettoyer les données du fichier .gexf pour intégrer les images en se connectant à la base de données EIDA :

a. Retirer les namespace des tags et modifier les chemins des images :

```
import xml.etree.ElementTree as ET

def enlever_prefixes_ns(root):
    # Fonction récursive pour retirer les namespaces des tags
    def strip_ns(elem):
```

```

        if '}' in elem.tag:
            elem.tag = elem.tag.split('}', 1)[1] # Retire namespace
avant '}'
        for child in elem:
            strip_ns(child)
    strip_ns(root)

def corriger_chemin_images_gexf(fichier_entree, fichier_sortie,
dossier_images="regions/"):
    tree = ET.parse(fichier_entree)
    root = tree.getroot()

    # 1) Nettoyer les namespaces
    enlever_prefixes_ns(root)

    # 2) Remettre les namespaces sur la racine
    ns_gexf = "http://gexf.net/1.3"
    ns_viz = "http://gexf.net/1.3/viz"
    root.attrib.clear()
    root.set("xmlns", ns_gexf)
    root.set("version", "1.3")
    root.set("xmlns:viz", ns_viz)
    root.set("xmlns:xsi",
"http://www.w3.org/2001/XMLSchema-instance")
    root.set("xsi:schemaLocation", "http://gexf.net/1.3
http://gexf.net/1.3/gexf.xsd")

    # 3) Modifier les chemins des images dans attvalues
    for node in root.findall("./node"):
        attvalues = node.find("attvalues")
        if attvalues is not None:
            for attvalue in attvalues.findall("attvalue"):
                if attvalue.attrib.get("for") == "image":
                    val = attvalue.attrib.get("value")
                    if val:
                        # Extraire juste le nom du fichier (sans
chemin absolu)
                        nom_fichier =
val.split("/")[-1].split("\\")[-1]
                        nouveau_chemin = dossier_images + nom_fichier

```



```

        attvalue.set("value", nouveau_chemin)

        print(f"Modifié image: {val} ->
{nouveau_chemin}")

    # 4) Sauvegarder le fichier corrigé
    tree.write(fichier_sortie, encoding='utf-8',
xml_declaration=True)
    print(f"Fichier corrigé sauvegardé sous : {fichier_sortie}")

if __name__ == "__main__":
    entree = "visualisation.gexf" # Fichier d'entrée
    sortie = "visualisation_corrige.gexf" # Fichier de sortie
    corriger_chemin_images_gexf(entree, sortie,
dossier_images="regions/")

```

- b. Convertir les chemins vers les images de la base de données en format IIIF :

```

import re
import xml.etree.ElementTree as ET

URL_PREFIX = "https://eida.obspm.fr/iiif/2/"
URL_SUFFIX = "/250,/0/default.jpg"

pat = re.compile(r"^(.*?\d{4})_(.+)\.jpg$", re.I) # base_page +
coords

def build_iiif_url(filename: str) -> str:
    """
    Ex. wit1_man191_0664_472,905,489,600.jpg
    -->
    https://eida.obspm.fr/iiif/2/wit1_man191_0664.jpg/472,905,489,600/250,/0/d
    efault.jpg
    """
    m = pat.match(filename.strip())
    if not m: # si le motif ne matche pas, on renvoie
le nom tel quel
        return filename
    base, coords = m.groups()
    return f"{URL_PREFIX}{base}.jpg/{coords}{URL_SUFFIX}"

```

```

def gexf_to_iiif(path_in: str, path_out: str):
    tree = ET.parse(path_in)
    root = tree.getroot()

    # (1) enlever d'éventuels préfixes ns0:, ns2:, ... pour faciliter la
    recherche
    def strip_ns(elem):
        if "}" in elem.tag:
            elem.tag = elem.tag.split("}", 1)[1]
        for child in elem:
            strip_ns(child)
    strip_ns(root)

    # (2) mise à jour des valeurs d'attribut image
    for av in root.findall("./attvalue[@for='image']"):
        old = av.get("value")
        if old:
            new = build_iiif_url(old.split("/")[-1]) # garde seulement le
nom du fichier
            av.set("value", new)
            print(f"{old} → {new}")

    # (3) sauvegarde
    tree.write(path_out, encoding="utf-8", xml_declaration=True)
    print(f"✅ Fichier transformé : {path_out}")

if __name__ == "__main__":
    gexf_to_iiif("visualisation_corrige.gexf",
"visualisation_iiif_corrige.gexf")

```

c. Enlever les liens internes des witnesses pour rendre le graphe plus lisible :

```

import xml.etree.ElementTree as ET

# Charger le fichier GEXF
tree = ET.parse("visualisation_iiif_corrige.gexf")
root = tree.getroot()

```

```

# Trouver la balise <graph> sans namespace
graph = root.find("graph")
if graph is None:
    raise ValueError("Balise <graph> introuvable (sans namespace).")

# Construire un dictionnaire node_id → cote
id_to_cote = {}

nodes_elem = graph.find("nodes")
if nodes_elem is None:
    raise ValueError("Balise <nodes> introuvable.")

for node in nodes_elem.findall("node"):
    node_id = node.attrib["id"]
    attvalues = node.find("attvalues")
    if attvalues is not None:
        for att in attvalues.findall("attvalue"):
            if att.attrib.get("for") == "cote":
                id_to_cote[node_id] = att.attrib["value"]

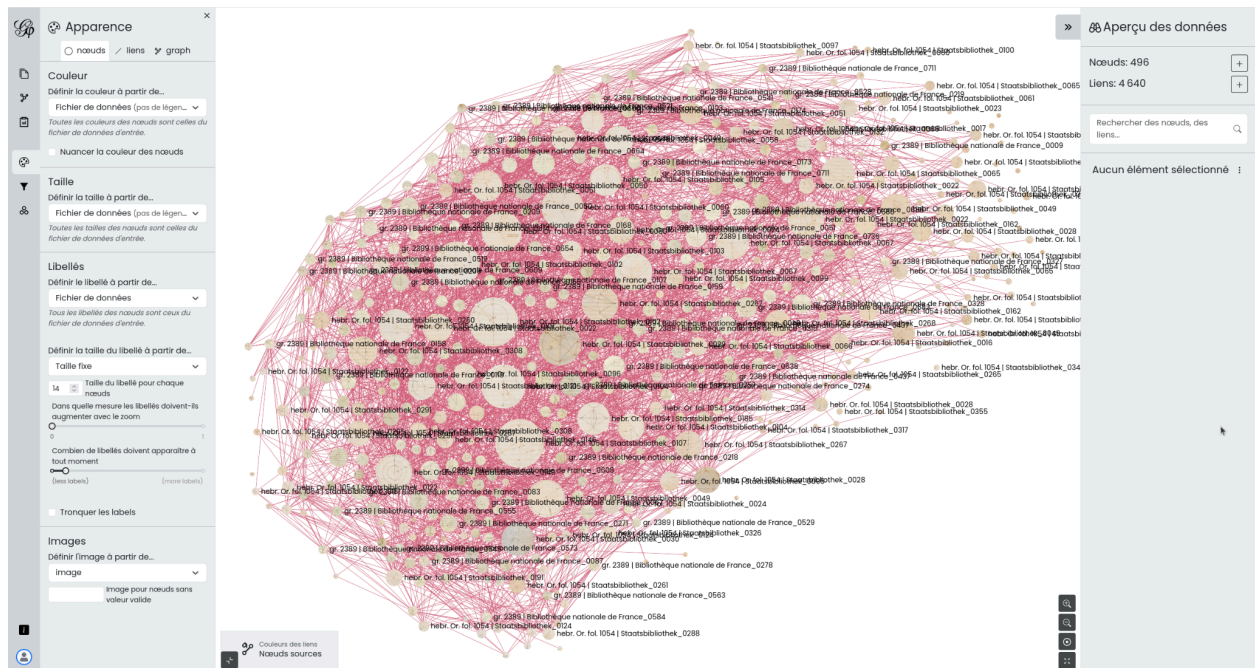
# Supprimer les <edge> entre nœuds du même manuscrit (même 'cote')
edges_elem = graph.find("edges")
if edges_elem is None:
    raise ValueError("Balise <edges> introuvable.")

removed_count = 0
edges = list(edges_elem.findall("edge"))
for edge in edges:
    source = edge.attrib["source"]
    target = edge.attrib["target"]
    if id_to_cote.get(source) == id_to_cote.get(target):
        edges_elem.remove(edge)
        removed_count += 1

# Sauvegarder le nouveau fichier
tree.write("fichier_sans_liens_internes.gexf", encoding="utf-8",
xml_declaration=True)
print(f"{removed_count} liens internes supprimés.")

```

13. Charger le fichier .gexf final dans Gephi Lite (<https://gephi.org/gephi-lite/>) :



14. Dans Apparence :

Apparence

nœuds
liens
graph

Couleur

Définir la couleur à partir de...

Fichier de données (pas de légende...)

Toutes les couleurs des nœuds sont celles du fichier de données d'entrée.

☐ Nuancer la couleur des nœuds

Taille

Définir la taille à partir de...

Fichier de données (pas de légende...)

Toutes les tailles des nœuds sont celles du fichier de données d'entrée.

Libellés

Définir le libellé à partir de...

Fichier de données

Tous les libellés des nœuds sont ceux du fichier de données d'entrée.

Définir la taille du libellé à partir de...

Taille fixe

14

Taille du libellé pour chaque nœuds

Dans quelle mesure les libellés doivent-ils augmenter avec le zoom

0
1

Combien de libellés doivent apparaître à tout moment

0
1

(less labels)
(more labels)

☐ Tronquer les labels

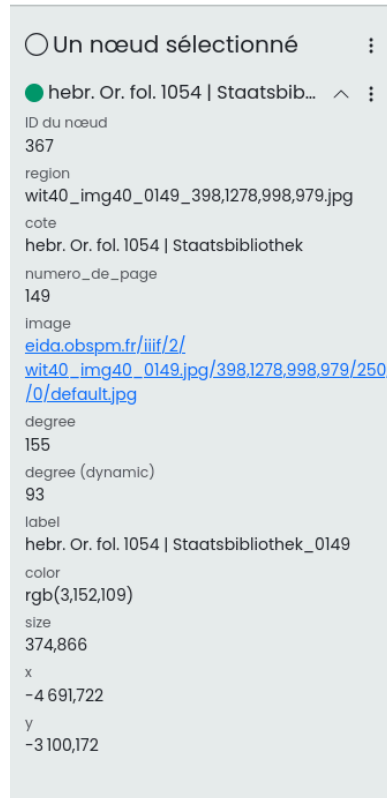
Images

Définir l'image à partir de...

image

Image pour nœuds sans valeur valide

- Pour l'apparence des nœuds, choisir "image" dans la section Images
 - Pour l'apparence des liens, choisir soit Nœuds source, soit Nœuds cible dans Couleurs. Finalement, cela a peu d'importance puisque nous avons enlevé les liens internes.
15. Désormais, lorsque nous cliquons sur un nœud, nous retrouvons cela sur le côté droit :



Visualisation de type “Bipartite graph”

L'intégralité du code, des images et des données de cette visualisation sont disponibles sur le repository github suivant : https://github.com/Cateatspython/bipartite_graph.

1. Faire un export csv des tables region_pairs des deux witnesses qui nous intéressent (wit75, wit76 et wit116). Nous voulons faire deux versions de la visualisation : une qui compare wit75 et wit 116 et l'autre qui compare wit76 et wit116.
2. Nettoyer les données des deux fichiers csv.
 - a. Importer les bibliothèques pandas et re :

```
import pandas as pd
import re
```

- b. Insérer le contenu du fichier csv dans un DataFrale witness :

```
witness = pd.read_csv('region_pair_75.csv', index_col=False)
```

- c. Créer une liste values avec les préfixes des regions que nous voulons garder dans le fichier csv (ici nous ne voulons les region pairs de wit76, wit75 et wit116 UNIQUEMENT) :

```
values = ["wit116", "wit75", "wit76"]
```

- d. Créer une variable `pattern` pour construire une expression régulière qui détectera si une chaîne de caractère commence par un ou plusieurs mots parmi les valeurs suivant d'un `_`.

```
pattern = r"^(" + "|".join([re.escape(v) + r"_" for v in values]) + ")"
```

- e. Ajouter deux variables `img_1` et `img_2` pour vérifier dans chaque ligne du DataFrame si la valeur des colonnes `img_1` et `img_2` commence par l'un des préfixes définis dans la variable `pattern` :

```
img1 = witness["img_1"].str.contains(pattern, na=False)
img2 = witness["img_2"].str.contains(pattern, na=False)
```

- f. Vérifier pour chaque ligne si la colonne `img_1` ET (uniquement si les deux conditions sont vraies) la colonne `img_2` contiennent le `pattern`.

```
chosen_witness = witness[img1 & img2]
```

- g. Pour supprimer les colonnes du document csv dont nous n'avons pas besoin, il faut d'abord créer une liste contenant les noms des colonnes à supprimer :

```
colonne_a_supprimer = ["regions_id_1", "regions_id_2", "created_at",
"updated_at", "category", "category_x", "is_manual"]
```

- h. Supprimer les colonnes de la liste `columns_suppression` grâce à la méthode `.drop()` :

```
new_witness = chosen_witness.drop(columns=colonne_a_supprimer)
```

- i. Insérer les données dans un nouveau fichier csv :

```
new_witness.to_csv('region_pairs_nettoyees_wit75.csv', index=False)
```

- j. Supprimer les types de similarité qui sont peu pertinentes à conserver :

```
similarity_type_a_supprimer = [4, 3]
```

```
similarity_filtrees=
new_witness[~new_witness['similarity_type'].isin(similarity_type_a_supprimer)]
```

```
similarity_filtrees.to_csv('data_wit75.csv', index=False)
```

- k. Créer une nouvelle colonne pour savoir si oui ou non la region pair est un lien interne :

```
similarity_filtrees = pd.read_csv('data_wit75.csv')

def get_prefix(val):
    if isinstance(val, str):
        return "_".join(val.split("_")[:2])
    return ""

# Extraire les préfixes
similarity_filtrees["prefix_1"] = similarity_filtrees["img_1"].apply(get_prefix)
similarity_filtrees["prefix_2"] = similarity_filtrees["img_2"].apply(get_prefix)

# Créer une nouvelle colonne booléenne : True si même préfixe (lien interne)
similarity_filtrees["lien_interne"] = similarity_filtrees["prefix_1"] == similarity_filtrees["prefix_2"]

# Supprimer les colonnes temporaires
similarity_filtrees.drop(columns=["prefix_1", "prefix_2"], inplace=True)

similarity_filtrees.to_csv('fichiers_liens_internes_wit75.csv', index=False)
```

- l. Garder uniquement les lignes dont la valeur de la colonne lien_interne est False et qui ne contiennent pas de wit75 dans img_1 ET img_2. Ici, nous voulons comparer wit76 et wit116 :

```
df = pd.read_csv('fichiers_liens_internes_wit76.csv')

# Garder seulement les lignes où lien_interne est False
# ET où 'wit76' n'apparaît ni dans img_1 ni dans img_2
df_filtered = df[
    (df['lien_interne'] == False) &
    (~df['img_1'].str.contains('wit75')) &
    (~df['img_2'].str.contains('wit75'))
]
```



```
# Sauvegarder le fichier filtré
df_filtered.to_csv('fichier_filtre_wit76.csv', index=False)
```

Note : Pour traiter le fichier csv du witness 75 pour le comparer avec witness 116 il suffit de changer la valeur wit75 par wit76.

m. Classer les différentes regions dans l'ordre des pages du witness.

```
import csv

def extraire_num_page(nom_fichier):
    # Exemple : "wit75_pdf75_0204_1099,845,404,398.jpg"
    # On veut extraire "0204"
    try:
        return int(nom_fichier.split('_')[2])
    except IndexError:
        return -1 # Si échec, valeur par défaut

# Lire le CSV
with open('FICHIERWIT116.csv', newline='', encoding='utf-8') as csvfile:
    reader = csv.DictReader(csvfile)
    lignes = list(reader)

# Trier selon le numéro de page extrait de img_1
lignes_triees = sorted(lignes, key=lambda row:
    extraire_num_page(row['img_1']))

# Écrire un nouveau CSV trié
with open('nouveau fichier116.csv', 'w', newline='', encoding='utf-8') as
csvfile:
    writer = csv.DictWriter(csvfile, fieldnames=reader.fieldnames)
    writer.writeheader()
    writer.writerows(lignes_triees)
```

3. Placer les fichiers csv dans un dossier csv_files/
4. Exporter les images des regions extraites sur la plateforme VHS. Les placer dans un dossier regions/
5. Créer un fichier HTML nommé bipartite_graph. Nous allons intégrer directement le JavaScript et le CSS dans ce fichier. Il s'agit d'une preuve-de-concept, il n'est donc pas obligatoire de créer trois fichiers

distincts car la visualisation ne sera pas intégrée telle quelle dans l'application. Nous allons utiliser la bibliothèque JavaScript [D3.js](#).

a. Dans le <head> du document importer la bibliothèque [D3.js](#) :

```
<script src="https://d3js.org/d3.v7.min.js"></script>
```

b. Toujours dans le <head> ajouter toute le script CSS dans une balise <style>.

```
<style>
  body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
  }
  #container {
    width: 100%;
    overflow-x: auto;
  }
  svg {
    width: 100%;
    border: 1px solid #ccc;
    background: #fafafa;
  }
  .node rect {
    stroke-width: 1;
  }
  .link {
    stroke: rgb(255, 0, 0);
  }
  text.score-text {
    font-size: 12px;
    fill: #333;
    pointer-events: none;
    user-select: none;
  }
  #filters {
    margin: 10px;
    text-align: center;
  }
  #filters label {
```

```

    margin-right: 15px;
    user-select: none;
}

#loadMore {
    padding: 10px 25px;
    font-size: 16px;
    cursor: pointer;
    border: none;
    background-color: #007bff;
    color: white;
    border-radius: 5px;
    transition: background-color 0.3s;
}

#loadMore:hover {
    background-color: #0056b3;
}

.column-label {
    font-size: 16px;
    font-weight: bold;
    fill: #333;
}
}
</style>

```

- c. Dans la balise body intégrer le titre de la visualisation en le centrant en haut de la page :

```
<h2 style="text-align: center;">Visualisation</h2>
```

- d. Créer une check-box pour choisir d'afficher ou non les scores de similarité :

```

<div id="filters">
    <label><input type="checkbox" id="toggle-score"> Afficher le
score</label>
</div>

```

- e. Intégrer la visualisation en elle-même dans un container dans le fichier html :

```

<div id="container">
    <div id="graph-wrapper">
        <svg></svg>
    </div>
</div>

```

```
</div>
</div>
```

- f. Intégrer un bouton “Afficher plus” pour gérer la quantité de region pour éviter de surcharger l’ordinateur à cause de la masse de données.

```
<div style="text-align: center; margin: 15px 0;">
  <button id="loadMore">Afficher plus</button>
</div>
```

- g. Ajouter la partie JavaScript dans une balise <script> à la fin du <body> :

- Créer des variables pour initialiser le svg :

```
const svg = d3.select("svg");
const g = svg.append("g");
```

- Créer des variables pour définir la mise en page.

```
const boxSize = 100;
const padding = 20;
const columnGap = 300;
```

- Créer une variable pour le fichier csv et une variable pour les images (en indiquant leur chemin) :

```
const file = "csv_files/wit75.csv";
const imagePath = "regions/";
```

- Créer des variables pour le chargement progressif des noeuds, la sélection d’un noeud et l’affichage ou non des scores.

```
const chunkSize = 50;
let displayedNodeCount = 0;
let allNodes = [];
let allLinks = [];
let showScores = false;
let selectedNodeId = null;
```

- Charger les données csv :

```
d3.csv(file).then(data => {
```

```
...
```

```
});
```

- Préparer les noeuds et les liens :

```
const nodesMap = new Map();

data.forEach(d => {
  d.similarity_type = String(d.similarity_type);
  if (d.lien_interne === "False") {
    d.id = `${d.img_1}-${d.img_2}`;
    allLinks.push(d);

    if (!nodesMap.has(d.img_1)) {
      nodesMap.set(d.img_1, { id: d.img_1, column: 0 });
    }
    if (!nodesMap.has(d.img_2)) {
      nodesMap.set(d.img_2, { id: d.img_2, column: 1 });
    }
  }
});
```

- Afficher progressivement les noeuds.

```
loadMore();
```

- Afficher un bouton pour afficher ou non les scores de similarité.

```
d3.select("#toggle-score").on("change", function () {
  showScores = this.checked;
  resetAndRender();
});
```

- Afficher un bouton pour afficher plus de noeuds :

```
d3.select("#loadMore").on("click", () => {
  loadMore();
  if (displayedNodeCount >= allNodes.length) {
    d3.select("#loadMore").style("display", "none");
  }
});
```

- Créer une fonction pour réinitialiser l'affichage de la première page de noeuds

```
function resetAndRender() {
  displayedNodeCount = chunkSize;
  d3.select("#loadMore").style("display", "inline-block");
  render(allNodes.slice(0, displayedNodeCount));
}
```

- Créer une fonction pour afficher progressivement des nouveaux noeuds :

```
function loadMore() {
  displayedNodeCount += chunkSize;
  if (displayedNodeCount > allNodes.length) displayedNodeCount =
allNodes.length;
  render(allNodes.slice(0, displayedNodeCount));
  if (displayedNodeCount >= allNodes.length) {
    d3.select("#loadMore").style("display", "none");
  }
}
```

- Créer une fonction pour placer les noeuds dans le svg :

```
function render(nodes) {
  const numColumns = 2;
  const contentWidth = (numColumns - 1) * columnGap + boxSize;
  const svgWidth = parseInt(svg.style("width"));
  const offsetX = (svgWidth - contentWidth) / 2;

  const positions = {};
  nodes.forEach((node, i) => {
    const x = offsetX + node.column * columnGap;
    const y = node.index * (boxSize + padding + 20) + padding + 40;
    positions[node.id] = { x, y };
  });
}
```

- Calculer dynamiquement la hauteur du SVG en fonction du nombre de noeuds affichés :

```
const maxIndex = d3.max(nodes, d => d.index);
const totalHeight = (maxIndex + 1) * (boxSize + padding + 20) +
padding + 60;
svg.attr("height", totalHeight);
```

- Ajouter des labels au dessus de chaque colonnes :

```
g.selectAll(".column-label").remove();
g.append("text")
  .attr("class", "column-label")
  .attr("x", offsetX + boxSize / 2)
  .attr("y", 20)
  .attr("text-anchor", "middle")
  .text("Witness 75");

g.append("text")
  .attr("class", "column-label")
  .attr("x", offsetX + columnGap + boxSize / 2)
  .attr("y", 20)
  .attr("text-anchor", "middle")
  .text("Witness 116");
```

- Afficher les noeuds du graphe, gérer leur interaction (en cliquant sur les noeuds) et mettre en évidence le noeud sélectionné et les autres noeuds qui sont liés à lui :

```
const nodeGroup = g.selectAll(".node")
  .data(nodes, d => d.id);

nodeGroup.exit().remove();

const nodeGroupEnter = nodeGroup.enter()
  .append("g")
  .attr("class", "node")
  .style("cursor", "pointer")
  .on("click", (event, d) => {
    if (selectedNodeId === d.id) {
      selectedNodeId = null;
    } else {
      selectedNodeId = d.id;
    }
    render(nodes);
  });

nodeGroupEnter.append("rect")
  .attr("width", boxSize)
```

```

    .attr("height", boxSize)
    .attr("fill", "#ddd")
    .attr("stroke", "#999");

nodeGroupEnter.append("image")
    .attr("width", boxSize)
    .attr("height", boxSize)
    .attr("href", d => imagePath + d.id);

nodeGroupEnter.append("text")
    .attr("x", boxSize / 2)
    .attr("y", boxSize + 12)
    .attr("text-anchor", "middle")
    .attr("font-size", "12px")
    .attr("fill", "#333")
    .text(d => d.id);

// === Modification ici pour entourer aussi les noeuds liés ===
let linkedNodes = new Set();
if (selectedNodeId !== null) {
    allLinks.forEach(link => {
        if (link.img_1 === selectedNodeId) linkedNodes.add(link.img_2);
        else if (link.img_2 === selectedNodeId)
linkedNodes.add(link.img_1);
    });
}

nodeGroup.merge(nodeGroupEnter)
    .attr("transform", d =>
`translate(${positions[d.id].x},${positions[d.id].y})`)
    .select("rect")
    .attr("stroke", d => (selectedNodeId === d.id ||
linkedNodes.has(d.id)) ? "red" : "#999")
    .attr("stroke-width", d => (selectedNodeId === d.id ||
linkedNodes.has(d.id)) ? 3 : 1)
    .attr("fill", d => (selectedNodeId === d.id ||
linkedNodes.has(d.id)) ? "#fdd" : "#ddd");

```

- Gérer le rendu des liens entre les noeuds (les liens sont de couleur pâle au début, quand on clique sur un noeud les liens

s'allument en rouge, leur épaisseur est en lien avec le score de similarité)

```
const filteredLinks = allLinks.filter(d =>
  d.img_1 in positions && d.img_2 in positions
);

const links = g.selectAll(".link")
  .data(filteredLinks, d => d.id);

links.exit().remove();

const linksEnter = links.enter()
  .append("line")
  .attr("class", "link");

links.merge(linksEnter)
  .attr("x1", d => positions[d.img_1].x + boxSize)
  .attr("y1", d => positions[d.img_1].y + boxSize / 2)
  .attr("x2", d => positions[d.img_2].x)
  .attr("y2", d => positions[d.img_2].y + boxSize / 2)
  .attr("stroke", d => {
    if (selectedNodeId === null) return "#ccc";
    return (d.img_1 === selectedNodeId || d.img_2 === selectedNodeId)
? "red" : "#ccc";
  })
  .attr("stroke-width", d => 1 + (parseFloat(d.score) / 100) * 5)
  .attr("opacity", d => {
    if (selectedNodeId === null) return 0.2;
    return (d.img_1 === selectedNodeId || d.img_2 === selectedNodeId)
? 1 : 0.1;
  });
```

- Ajouter les scores de similarité sur les liens quand la fonctionnalité est activée :

```
g.selectAll(".score-text").remove();
if (showScores) {
  g.selectAll(".score-text")
```

```

        .data(filteredLinks)
        .enter()
        .append("text")
        .attr("class", "score-text")
        .attr("x", d => (positions[d.img_1].x + positions[d.img_2].x +
boxSize) / 2)
        .attr("y", d => (positions[d.img_1].y + positions[d.img_2].y) / 2
+ boxSize / 2)
        .attr("text-anchor", "middle")
        .text(d => parseFloat(d.score).toFixed(1));
    }
}

```

6. Pour afficher le résultat, il suffit à présent d'ouvrir le fichier HTML avec Live Server en local.